



Security analysis and improvement of a public auditing scheme for secure data storage in fog-to-cloud computing

Wenfang Zhang^{a,*}, Heng Jiao^a, Zhuoqun Yan^{a,*}, Xiaomin Wang^a,
Muhammad Khurram Khan^b

^a School of Information Science and Technology, Southwest Jiaotong University, Chengdu, Sichuan 610031, China

^b Center of Excellence in Information Assurance, College of Computer & Information Sciences, King Saud University, Riyadh 11653, Saudi Arabia

ARTICLE INFO

Article history:

Received 3 July 2022

Revised 7 October 2022

Accepted 17 November 2022

Available online 20 November 2022

Keywords:

Cloud storage

IoT

Data integrity checking

Fog computing

Security analysis

ABSTRACT

Human society has drastically changed as a result of the widespread application of Internet of Things (IoT) technology, particularly in various industrial fields. In spite of the advantages brought by IoT services, the underlying security challenges cannot be underestimated. Cloud-based IoT data collection poses a number of challenges, including how to ensure its integrity. In order to address this issue, Tian et al. have proposed a fog-to-cloud computing-based public auditing scheme in IoT scenarios, which provides the data-privacy preserving mechanism and tag transformation strategy. However, in this paper, we show that Tian et al.'s scheme fails to achieve soundness, an essential security property, by giving two attacks. In the first attack, the malicious cloud server can delete all the data and then deceive the Third Party Auditor (TPA) into believing that the data is well-preserved. In the second attack, the malicious cloud server can modify the outsourced data and then deceive TPA into believing that all the data is kept intact. We further provide a simple but effective solution for Tian et al.'s scheme to resist the above-mentioned attacks. Security analysis and performance evaluation are also given to demonstrate the robustness and high efficiency of the improved scheme.

© 2022 Elsevier Ltd. All rights reserved.

1. Introduction

Internet of Things (IoT) consists of automatic sensors embedded in various physical devices. By utilizing and analyzing the device data collected from widespread distributed sensors, IoT technology makes it possible for people to interact with the objects around us and provides us a more convenient life. While enjoying the convenience brought by IoT technology, what follows is the challenge of transforming and processing an unprecedented amount of data generated constantly by geographically distributed IoT devices. Considering the advantage of tremendous computation and storage resources, cloud computing has been applied into sorts of IoT scenarios, such as data analysis, data mining and data storage, as a promising technology solution. However, due to the inherently high centralized feature of cloud computing itself, it is inevitable that all the outsourced data need to be centrally transmitted to the remote center server via the network. As a result, while responding to multiple service requests simultaneously, regional network

congestion occurs from time to time, which leads to slow response of service requests and low-quality service experience.

Therefore, fog computing, as an important complement of cloud computing, is recently attracting considerable interest of researchers. With computing resources moving from distant cloud server to fog nodes located at the edge of data source, fog computing is capable of processing data in real time and responding service request with low time-delay under special application scenarios, (such as Driverless car, Augmented reality, etc.). In a typical fog-to-cloud computing architecture, all the original data generated by kinds of distributed sensors will firstly be sent to the local fog nodes for data pre-processing, and then outsourced to the remote cloud server for further analysis and storage.

In spite of all aspects of promising prospects, fog-to-cloud computing is not free of security challenges. How to ensure the integrity of outsourced data stored in the untrusted servers is a fundamental issue. For chasing financial benefit or protecting reputation itself, cloud servers may try to convince users that all the outsourced data is remained intact, while parts of data possibly has been tampered or corrupted. At the very worst case, cloud server can even delete the data user rarely accessed for saving its storage resource. To guarantee the integrity of data in the cloud, there is a vast amount of literatures on Data Integrity Checking.

* Corresponding authors.

E-mail addresses: wfzhang@swjtu.edu.cn (W. Zhang), jiaoheng_x@163.com (H. Jiao), yzq0622@my.swjtu.edu.cn (Z. Yan), xmwang@swjtu.edu.cn (X. Wang), mkhurram@KSU.EDU.SA (M.K. Khan).

Ateniese et al. (2007) proposed a data integrity checking protocol in untrusted remote cloud environment based on the notion of provable data possession (PDP). With the help of using homomorphic verifiable tag (HVT), the protocol enables data owners perform data integrity checking without downloading their original data, which greatly decreases the communication overhead. At the same time, Juels and Kaliski (2007) explored the proofs of retrievability (POR) scheme to achieve data integrity checking for large files in cloud. This method makes use of error correction and random permutation so that data owner can detect the changes of stored files. However, these schemes can only support static files for integrity verifying. When encountering the situation that the stored files in cloud are completely dynamic, adaptive changes are needed for these schemes. To support data dynamics, various approaches are presented (Chen et al., 2020; He et al., 2021; Jiang et al., 2016; Shen et al., 2017; Sookhak et al., 2018; Wang et al., 2011; Shacham and Waters, 2013). Many practical data structures applied in computer storage field have been reused to design data integrity checking protocol, such as divide and conquer tables, Merkle hash tree and doubly linked info table. Moreover, focusing on dynamic sensitive data sharing scenario, a mass of data integrity checking protocols with privacy preserving and group user revocation have been proposed by researchers. To protect user identity privacy from cloud server, Li et al. (2019) adopted the core idea of Fuzzy signature to design their scheme. Shen et al. (2019) made use of the idea in sanitizable signature to hide sensitive information from other group members. By introducing a third party, named sanitizer, into system, the sensitive information and its verification tags can be converted into uniform format. In Sun et al. (2020); Tian et al. (2019); Wang et al. (2014); Zhao et al. (2019), authors also took different methods to achieve user information privacy preserving, like random mask encryption, ring signature and so on. Moreover, to handle the problem of user revocation in dynamic group under cloud data sharing scenario, Wang et al. (2015) proposed a novel scheme, named Panda. By using the proxy re-signatures, Panda greatly reduces the communication and computation overhead on the user side. From another perspective, Zhang et al. (2020) raised an efficient solution to realize user revocation by updating the private keys of non-revoked users. Compared to Panda, the computation overhead in the whole system has been further reduced.

Nevertheless, all the aforementioned schemes are not useable in practical IoT scenarios. The main limitation of those protocols is two-fold. firstly, unlike data generated by certain users in cloud environment, the data sources are various sensors in the wireless network. Then, difference in the strategy of tagging between fog-to-cloud computing paradigm and cloud computing paradigm produces. Secondly, the fog nodes have not been taken into consideration during the whole stage of data processing in former protocols, while under real IoT scenarios fog nodes play an irreplaceable role, such as caching necessary local content, aiding in pre-processing for real data stream. Therefore, it is imperative and urgent to design a suitable data integrity checking protocol in fog-to-cloud computing for real IoT scenarios.

Tian et al. (2019) first proposed a public auditing scheme for fog-to-cloud computing, which both achieves the tag transforming strategy between the mobile sinks (seen as data sources) and fog nodes and perfect data privacy-preserving against TPA. To tackle the problem that authenticatable tag of metadata generated by different mobile sinks with different keys, they designed a novel tag transforming strategy, which converts tags produced by mobile sinks to the ones produced by fog nodes. Specially, fog nodes will resign the semi-finished authenticatable tag received from mobile sinks and then forward it to cloud sever together with its personal authenticatable tags. On the other hand, a zero-knowledge proof mechanism is introduced during the verification phase. Based on

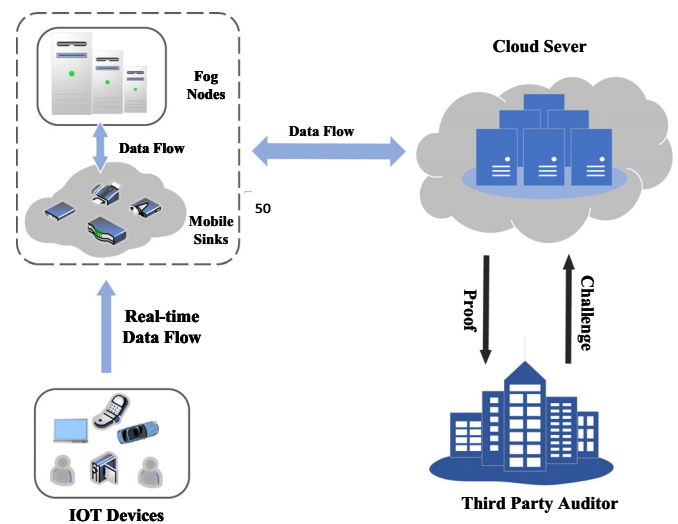


Fig. 1. System model.

the hardness of discrete logarithm problem, the identity privacy of all data generators is perfectly protected from cloud server.

In this paper, we investigate the details of Tian et al. (2019) and analyze its security. Moreover, we point out two potential attacks, and then corresponding countermeasures are given to prevent those two attacks. Our contributions can be summed up as follows. We show that the cryptographical construction in Tian et al. (2019) fails to achieve the fundamental security property in data integrity checking schemes. We demonstrate that their scheme is vulnerable to a malicious server which is capable of launching two grave attacks, i.e., data replacing attack and data deleting attack. In the first attack, while part of challenge data blocks is corrupted, a malicious server can use some other well-maintained data blocks to forge a valid proof, and then pass the verification. In the latter attack, a malicious server can even delete all the outsourced data and only store the parameters that are needed. Furthermore, those two attacks launched by a malicious server will not be detected by TPA or other auditors, and data owner may always believe that data remains intact in cloud until it is retrieved. We then present an improved protocol to tackle the vulnerability discussed above and then prove that our new protocol is secure against above-mentioned attacks.

2. Review of Tian et al.'s protocol

In this section, we provide a comprehensive overview of the Tian et al.'s protocol including the system model.

2.1. System model

As shown in Fig. 1, there are five entities in this protocol: IoT devices, mobile sinks, fog nodes, cloud server and third-party auditor (TPA). IoT devices are seen as the sensor data producers, which produce data and send them to the mobile sinks. Mobile sinks, responsible for collecting data, are considered as the actual data sources, which will also generate semi authenticatable tags for received data before forwarding them to fog nodes. Fog nodes play two roles. They first verify the correctness of the received data before further processing and then generate the final authenticatable tag on the basis of semi authenticatable tag. All the data with final authenticatable tag will be transmitted to the cloud servers, which provide storage and computing services in a scalable way. TPA plays a role in checking the integrity of the outsourced data regularly on behalf of the data owner.

2.2. Tian et al.'s protocol

Let G and G_T be two multiplicative cyclic groups with the same prime order p . g and u are two generators of G . Let $e : G \times G \rightarrow G_T$ be a bilinear map, and $H_1 : \{0, 1\}^* \rightarrow Z_p^*$, $H_2 : \{0, 1\}^* \rightarrow G$, $H_3 : G_T \rightarrow Z_p^*$ are three hash functions. There are w mobile sinks and t fog nodes in the protocol, denoted as $S = \{s_i, i = 1, 2, \dots, w\}$ and $F = \{f_v, v = 1, 2, \dots, t\}$ respectively. Tian et al. (2019) is composed of six algorithms.

KeyGen. Each mobile sink $S_i (i = 1, 2, \dots, w)$ generates its key pair $(spk_i = g^{\frac{1}{d_i}}, ssk_i = a_i)$, where a_i is randomly chosen from Z_p^* . Similarly, each fog node $f_v (v = 1, 2, \dots, t)$ generates its key pair $(fpk_v = g^{b_v}, fsk_v = b_v)$, where b_v is randomly chosen from Z_p^* .

TagGenMS. Each mobile sink s_i generates the semi-authenticatable tags $\sigma'_{i,j}$ for data block $m_{i,j} (j = 1, 2, \dots, n_i)$ with its private key ssk_i as follows:

$$\sigma'_{i,j} = (u^{H_1(id_{i,j})} g^{m_{i,j}})^{ssk_i} \quad (1)$$

where $id_{i,j}$ is the unique identifier of each data block $m_{i,j}$

TagGenF. Before further processing the data block $m_{i,j}$ received from the i th mobile sink, the corresponding fog node $f_v (v = 1, 2, \dots, t)$ checks its integrity first as follows:

$$e(\sigma'_{i,j}, spk_i) = e(u^{H_1(id_{i,j})} \cdot g^{m_{i,j}}, g) \quad (2)$$

If it does not hold, the fog node f_v refuses to process $m_{i,j}$ and $\sigma'_{i,j}$; otherwise, f_v resigns $\sigma'_{i,j}$ to generate the final authenticatable tag with its private key as follows:

$$\sigma_{i,j} = (\sigma'_{i,j})^{fsk_v} \quad (3)$$

also, f_v generates an authenticatable signature $\vartheta_{i,j}$ for both $m_{i,j}$ and $\sigma_{i,j}$ as

$$\vartheta_{i,j} = (H_2(\sigma_{i,j} \| m_{i,j}))^{fsk_v} \quad (4)$$

Finally, all the data blocks and their final authenticatable tags are transmitted to the cloud server. Upon receiving data block $m_{i,j}$, the cloud server checks the correctness of its signature first by using the following equation:

$$e(\vartheta_{i,j}, g) = e(H_2(\sigma_{i,j} \| m_{i,j}), fsk_v) \quad (5)$$

If the equation holds, the cloud server accepts and stores $m_{i,j}$ and its final authenticatable tag $\sigma_{i,j}$; otherwise, the cloud server asks the corresponding fog node for retransmitting $m_{i,j}$ and $\sigma_{i,j}$. Moreover, for each data block $m_{q,j}$ created by fog node itself where $q = w + v$, f_v also generates a final authenticatable tag as

$$\sigma_{q,j} = (u^{H_1(id_{q,j})} \cdot g^{m_{q,j}})^{fsk_v} \quad (6)$$

Then, f_v sends these data and their tags to the cloud. For each data block $m_{q,j}$, the cloud server checks its integrity first by using the following equation:

$$e(\sigma_{q,j}, g) = e(u^{H_1(id_{q,j})} g^{m_{q,j}}, spk_v) \quad (7)$$

If the equation holds, the cloud server accepts and stores $m_{q,j}$ and its final authenticatable tag $\sigma_{q,j}$; otherwise, the cloud server asks the corresponding fog node for retransmitting $m_{q,j}$ and $\sigma_{q,j}$.

Challenge. To check the integrity of the outsourced data, TPA generates the challenge

$$chal = \{\mathbb{C}, \mathbb{R}, \{\Phi_v\}_{v \in [1, t]}, \{\theta_i\}_{i \in [1, w]}, \zeta, \Lambda\}$$

as follows:

- 1) The TPA first randomly selects τ different data blocks from all the data blocks as a challenge set $\mathbb{C} = \{ID_1, ID_2, \dots, ID_\tau\}$, and a corresponding set of random numbers $\mathbb{R} = \{r_k \in Z_p^* | k \in \mathbb{C}\}$.

- 2) Next the TPA generates special challenges φ_v and θ_i for fog node f_v and mobile sink s_i respectively. More specially, the TPA chooses a random number $\rho \in Z_p^*$, and for each fog node $f_v (v \in [1, t])$ computes $\varphi_v = e(g, fpk_v)^\rho$; For each mobile sink $s_i (i \in [1, w])$, the TPA computes $\theta_i = spk_i^\rho = g^{\frac{\rho}{d_i}}$. And for all the fog nodes, the TPA also computes a common identifier $\zeta = g^\rho$.

- 3) Then the TPA uses all calculations as explained above to generate a knowledge proof (denoted by Λ) as

$$\Lambda = POK\{(\rho) : \varphi_1 \wedge \dots \wedge \varphi_t \wedge \theta_1 \wedge \dots \wedge \theta_w \wedge \zeta\}$$

- 4) Finally, the TPA sends the challenge $chal$ to the cloud server.

PoofGen. Upon receiving the challenge information from the TPA, the cloud server first verifies the knowledge proof Λ . if Λ is valid, the cloud server generates the response for proving the correctness of data storage as follows:

- 1) For $i \in [1, w + t]$ and $id_{i,j} \in \mathbb{C}_i$, the cloud server computes the tag proof Φ_i as $\Phi_i = \prod_{j \in \mathbb{C}_i} \sigma_{i,j}^{r_{i,j}}$, and the data proof as $\Omega_i = \sum_{j \in \mathbb{C}_i} m_{i,j} \cdot r_{i,j}$, where $\mathbb{C}_i$ is the set of challenged blocks handled by the i th entity, $id_{i,j}$ is the j th element of $\mathbb{C}_i$, and $r_{i,j}$ is the corresponding random number.
- 2) For $v \in [1, t]$, the cloud server further aggregates the data proof above as $\Psi = \sum_{\mathbb{C}_i \subset \mathbb{C}_v} \Omega_i$, where $\mathbb{C}_i$ is the set of challenged blocks handled by the v th fog node.
- 3) The cloud server derives the response Θ from the tag proof Φ_i , the data proof Ω_i and the aggregated data proof Ψ as

$$\Theta = H_3\left(\left(\prod_{i=1}^w e(\Phi_i, \theta_i)\right) \cdot \prod_{v=1}^t e(\Phi_v, \zeta) \cdot \prod_{v=1}^t \varphi_v^{-\Psi_v}\right) \quad (8)$$

- 4) Finally, the cloud server sends the response Θ as the proof of possession of data storage back to the TPA.

Verify. The TPA verifies the correctness of Θ by checking the following equation:

$$H_v = \sum_{j \in \mathbb{C}_v} (r_{v,j} \cdot H_1(id_{v,j})) \quad (9)$$

$$\Theta = H_3\left(\prod_{v=1}^t e(u^{H_v}, fpk_v^\rho)\right) \quad (10)$$

It outputs TRUE if all the equation holds; otherwise, FALSE.

3. Security analysis of Tian et al.'s protocol

In this section, we first demonstrate the weakness in scheme of Tian et al. Then, we present two potential attacks that pose a serious threat to the security feature of soundness owing to an improper-designed tag generation strategy. In the first attack, while parts of challenged blocks are corrupted, a malicious cloud server can forge a valid authenticatable tag of any fog node without its private key and then generate a forgeable response obtained from some tamper data blocks to pass the verification. In the second attack, a malicious cloud server only stores some parameters but not the outsourced data and then convinces the TPA that all the data is preserved intact. Consequently, Tian et al. (2019) cannot satisfy the security requirements of public auditing schemes any longer. In the following, we explain the weakness and the two attacks in detail.

3.1. Weakness

In Tian et al. (2019), the authenticatable tag $\sigma_{q,j}$ is generated as $\sigma_{q,j} = (u^{H_1(id_{q,j})} \cdot g^{m_{q,j}})^{fsk_v}$. However, due to the improper construction of authenticatable tag, i.e., a specific generator g which has been used to generate public keys of all the entities (mobile sinks and fog nodes) is reused to generate authenticatable tag. We found that a malicious sever can obtain useful knowledge from the authenticatable tag to reconstruct any valid authenticatable tag without corresponding private key. By manipulating this forged authenticatable tag, the cloud server can forge a valid response and launches its strikes.

We show how a malicious cloud server completes its preparatory work before its attack as follows:

- 1) For any f_v , the cloud server chooses a block $m_{q,j}$ and $\sigma_{q,j}$ to compute

$$\begin{aligned}\beta'_v &= \sigma_{q,j} \cdot fpk_v^{-m_{q,j}} \\ &= (u^{H_1(id_{q,j})} \cdot g^{m_{q,j}})^{fsk_v} \cdot (g^{fsk_v})^{-m_{q,j}} \\ &= u^{H_1(id_{q,j}) \cdot fsk_v}\end{aligned}\quad (11)$$

- 2) Then computes

$$\begin{aligned}\beta_v &= (\beta'_v)^{(H_1(id_{q,j}))^{-1}} \\ &= u^{H_1(id_{q,j}) \cdot fsk_v \cdot (H_1(id_{q,j}))^{-1}} \\ &= u^{fsk_v}\end{aligned}\quad (12)$$

Now, the cloud server stores $\beta_v = u^{fsk_v}$, and in the future it is capable of forging any valid response.

3.2. Attack 1

Tian et al. have claimed that the cloud server cannot launch the forge attack, i.e., the cloud server cannot pass the verification with modified data blocks and tags. However, we show that the cloud server can forge a valid response with the help of β_v and any tempered data block $m'_{i,j}$.

Upon receiving the challenge $chal$, the cloud server can generate a valid response Θ' through the following steps.

- 1) Assume that $\mathbb{C}_v$, ($1 \leq v \leq t$) is the identifier set of the challenged blocks resigned or generated by the v th fog node. For $id_{i,j} \in \mathbb{C}_v$, the cloud server computes

$$\sigma'_{i,j} = (\beta_v)^{H_1(id_{i,j})} \cdot fpk_v^{m'_{i,j}} = (u^{H_1(id_{i,j})} \cdot g^{m'_{i,j}})^{fsk_v} \quad (13)$$

where $i \in [1, w+t]$ and $v \in [1, t]$.

- 2) Computes the tag proof $\Phi'_i = \prod_{j \in \mathbb{C}_i} (\sigma'_{i,j})^{r_{i,j}}$ and the data proof

$$\Omega'_i = \sum_{j \in \mathbb{C}_i} (m'_{i,j} \cdot r_{i,j}), \text{ where } i \in [1, w+t]. \text{ Computes the aggregated data proof } \Psi'_v = \sum_{i \in \mathbb{C}_v} \Omega'_i \text{ where } v \in [1, t].$$

- 3) Finally, the cloud server outputs the response

$$\Theta' = H_3 \left(\left(\prod_{i=1}^w e(\Phi'_i, \zeta) \right) \cdot \prod_{v=1}^t e(\Phi'_v, \zeta) \cdot \prod_{v=1}^t \varphi_v^{-\Psi'_v} \right) \quad (14)$$

The response Θ' can pass the verification since

$$\Theta' = H_3 \left(\left(\prod_{i=1}^w e(\Phi'_i, \zeta) \right) \cdot \prod_{v=1}^t e(\Phi'_v, \zeta) \cdot \prod_{v=1}^t \varphi_v^{-\Psi'_v} \right)$$

$$\begin{aligned}&= H_3 \left(\left(\prod_{i=1}^w e \left(\prod_{j \in \mathbb{C}_i} (\sigma'_{i,j})^{r_{i,j}}, g^\rho \right) \cdot \prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_{w+v}} (\sigma'_{w+v,j})^{r_{w+v,j}}, g^\rho \right) \cdot \prod_{v=1}^t e(g, fpk_v)^{-\Psi'_v \cdot \rho} \right) \right) \\&= H_3 \left(\frac{\prod_{v=1}^t \prod_{i \in [1, w+t]} e \left(\prod_{j \in \mathbb{C}_i} (u^{H_1(id_{i,j})} \cdot g^{m'_{i,j}})^{b_v \cdot r_{i,j}}, g^\rho \right)}{\prod_{v=1}^t e(g, g^{b_v})^{\Psi'_v \cdot \rho}} \cdot \prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_{w+v}} (u^{H_1(id_{i,j})} \cdot g^{m'_{w+v,j}})^{b_v \cdot r_{w+v,j}}, g^\rho \right) \right) \\&= H_3 \left(\frac{\prod_{v=1}^t \prod_{i \in [1, w+t]} e \left(\prod_{j \in \mathbb{C}_i} (u^{r_{i,j} \cdot H_1(id_{i,j})} \cdot g^{r_{i,j} \cdot m'_{i,j}}), g^{b_v \cdot \rho} \right)}{\prod_{v=1}^t e \left(g^{\sum_{k \in \mathbb{C}_v} r_k \cdot m'_k}, g^{b_v \cdot \rho} \right)} \right) \\&= H_3 \left(\frac{\prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_v} (u^{r_{v,j} \cdot H_1(id_{v,j})} \cdot g^{\sum_{k \in \mathbb{C}_v} r_k \cdot m'_k}), g^{b_v \cdot \rho} \right)}{\prod_{v=1}^t e \left(g^{\sum_{k \in \mathbb{C}_v} r_k \cdot m'_k}, g^{b_v \cdot \rho} \right)} \right) \\&= H_3 \left(\prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_v} u^{r_{v,j} \cdot H_1(id_{v,j})}, g^{b_v \cdot \rho} \right) \right) \\&= H_3 \left(\prod_{v=1}^t e(u^{H_v}, fsk_v^\rho) \right) \quad (15)\end{aligned}$$

As a result, the cloud server launches its forge attack successfully with the tampered data block $m'_{i,j}$ and deceives the TPA that all the outsourced data remains well.

3.3. Attack 2

In this attack, we show that while all the outsourced data has been deleted, the cloud server can still forge a valid response Θ' with the help of β_v . More concretely, upon receiving the challenge, the cloud server can generate a valid response through the following steps.

- 1) For each $id_{i,j} \in \mathbb{C}_v$, the cloud server computes

$$\sigma'_{i,j} = (\beta_v)^{H_1(id_{i,j})} = u^{H_1(id_{i,j}) \cdot fsk_v} \quad (16)$$

where $i \in [1, w+t]$ and $v \in [1, t]$.

- 2) Computes the tag proof $\Phi'_i = \prod_{j \in \mathbb{C}_i} (\sigma'_{i,j})^{r_{i,j}}$ where $i \in [1, w+t]$.

Computes

$$\Theta' = H_3 \left(\left(\prod_{i=1}^w e(\Phi'_i, \zeta) \right) \cdot \prod_{v=1}^t e(\Phi'_v, \zeta) \right) \quad (17)$$

The response Θ' can pass the verification since

$$\begin{aligned}\Theta' &= H_3 \left(\left(\prod_{i=1}^w e(\Phi'_i, \zeta) \right) \cdot \prod_{v=1}^t e(\Phi'_v, \zeta) \right) \\&= H_3 \left(\left(\prod_{i=1}^w e \left(\prod_{j \in \mathbb{C}_i} (\sigma'_{i,j})^{r_{i,j}}, g^\rho \right) \cdot \prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_{w+v}} (\sigma'_{w+v,j})^{r_{w+v,j}}, g^\rho \right) \right) \right) \\&= H_3 \left(\frac{\prod_{v=1}^t \prod_{i \in [1, w+t]} e \left(\prod_{j \in \mathbb{C}_i} u^{H_1(id_{i,j}) \cdot r_{i,j} \cdot b_v}, g^\rho \right)}{\prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_{w+v}} (u^{H_1(id_{w+v,j}) \cdot b_v \cdot r_{w+v,j}}, g^\rho \right)} \right)\end{aligned}$$

$$\begin{aligned}
&= H_3 \left(\prod_{v=1}^t \prod_{\substack{C_i \in \mathbb{C}_v \\ i \in [1, w+t]}} e \left(\prod_{j \in C_i} u^{H_1(id_{i,j}) \cdot r_{i,j}}, g^{\rho \cdot b_v} \right) \right) \\
&= H_3 \left(\prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_v} u^{r_{v,j} \cdot H_1(id_{v,j})}, g^{b_v \cdot \rho} \right) \right) \\
&= H_3 \left(\prod_{v=1}^t e(u^{H_v}, fsk_v^\rho) \right) \quad (18)
\end{aligned}$$

Consequently, the cloud server can only spend a piece of storage resource to store the parameter u^{fsk_v} needed in proof forging instead of the outsourced data. Therefore, this protocol fails to achieve the security property.

4. An improved protocol

In this section, we intend to revamp [Tian et al. \(2019\)](#) which suffers security attacks. We modify the way of generating data blocks authenticatable tag to avert degeneration of the effect of data encryption, such that the adversary sever can no longer obtain β_v by leveraging authenticatable tag. Then we demonstrate how the protocol with improved authenticatable tag resists the attacks.

4.1. Improved protocol details

Intuitively, [Tian et al. \(2019\)](#) can be amended in a variety of way to achieve a standard level of security. On the one hand, considering that both attacks are executed successfully with the help of β_v , one way to toughen security soundness is to modify the way of generating data blocks authenticatable tag, which prevents any adversary server from getting useful knowledge from authenticatable tag to forge valid response. On the other hand, those attacks can also be avoided if we change the sequencing of basic protocol execution, i.e., let TPA derive the response Θ from Φ_i , Ω_i and Ψ instead the cloud server. Then TPA executes the **Verify** algorithm to verify the correctness of proof from the cloud server.

In the latter case, however, changing the sequencing of protocol execution may contradict with the original idea in basic protocol. The cloud server is required to transfer a great deal of proof information and TPA has to consume a lot of computing resources, which increases the complexity of the whole system. Therefore, we give a solution to the weakness by modifying the way of generating data blocks authenticatable tag while both the communication costs and computational overhead stay the same as the initial protocol. We describe the details related to concrete improvement in the following.

As the major differences between the new protocol and the initial protocol are the **TagGenMS**, **TagGenF** and **Verify** algorithms, other algorithms are omitted because they are the same as the initial protocol.

TagGenMS. Each mobile sink s_i generates the semi-authenticatable tags $\sigma_{i,j}$ for data block $m_{i,j}$ ($j = 1, 2, \dots, n_i$) with its private key ssk_i as

$$\sigma_{i,j} = (H_2(id_{i,j}) \cdot u^{m_{i,j}})^{ssk_i} \quad (19)$$

where $id_{i,j}$ is the unique identifier of each data block $m_{i,j}$

TagGenFS. Similarly, each fog node generates its authenticatable tags $\sigma_{q,j}$ as

$$\sigma_{q,j} = (H_2(id_{q,j}) \cdot u^{m_{q,j}})^{fsk_q} \quad (20)$$

where $id_{q,j}$ is the unique identifier of each data block $m_{q,j}$

Verify. The TPA verifies the correctness of Θ by checking the following equations:

$$H_v = \prod_{j \in \mathbb{C}_v} (H_2(id_{v,j}))^{r_{v,j}} \quad (21)$$

$$\Theta = H_3 \left(\prod_{v=1}^t e(u^{H_v}, fsk_v^\rho) \right) \quad (22)$$

it outputs TRUE if both equations hold; otherwise, FALSE.

We then show the correctness of above equations in the **Verify** algorithm.

$$\begin{aligned}
\Theta &= H_3 \left(\left(\prod_{i=1}^w e(\Phi_i, \theta_i) \right) \cdot \prod_{v=1}^t e(\Phi_v, \zeta) \cdot \prod_{v=1}^t \varphi_v^{-\Psi_v} \right) \\
&= H_3 \left(\prod_{i=1}^w e \left(\prod_{j \in C_i} (\sigma_{i,j})^{r_{i,j}}, g^{\frac{\rho}{a_i}} \right) \cdot \prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_{w+v}} (\sigma_{w+v,j})^{r_{w+v,j}}, g^\rho \right) \cdot \prod_{v=1}^t e(g, fsk_v)^{-\Psi_v \cdot \rho} \right) \\
&= H_3 \left(\frac{\prod_{v=1}^t \prod_{\substack{C_i \in \mathbb{C}_v \\ i \in [1, w]}} e \left(\prod_{j \in C_i} (H_1(id_{i,j}) \cdot u^{m_{i,j}})^{b_v \cdot r_{i,j}}, g^\rho \right)}{\prod_{v=1}^t e(g, g^{b_v})^{\Psi_v \cdot \rho}} \cdot \prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_{w+v}} (H_1(id_{v,j}) \cdot u^{m_{w+v,j}})^{b_v \cdot r_{w+v,j}}, g^\rho \right) \right) \\
&= H_3 \left(\frac{\prod_{v=1}^t \prod_{\substack{C_i \in \mathbb{C}_v \\ i \in [1, w+t]}} e \left(\prod_{j \in C_i} (H_1(id_{i,j}))^{r_{v,j}} \right) \cdot u^{r_{i,j} \cdot m_{i,j}}, g^{b_v \cdot \rho} \right)}{\prod_{v=1}^t e(g^{k \in \mathbb{C}_v}, g^{b_v \cdot \rho})} \\
&= H_3 \left(\frac{\prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_v} (H_1(id_{i,j}))^{r_{v,j}} \cdot u^{r_{i,j} \cdot m_{i,j}}, g^{b_v \cdot \rho} \right)}{\prod_{v=1}^t e \left(g^{\sum_{k \in \mathbb{C}_v} r_k \cdot m_k}, g^{b_v \cdot \rho} \right)} \right) \\
&= H_3 \left(\prod_{v=1}^t e \left(\prod_{j \in \mathbb{C}_v} H_1(id_{i,j})^{r_{v,j}}, g^{b_v \cdot \rho} \right) \right) \\
&= H_3 \left(\prod_{v=1}^t e(H_v, fsk_v^\rho) \right) \quad (23)
\end{aligned}$$

4.2. Security analysis

In this subsection, we prove that our improved scheme is secure against data replacing attack and data deleting attack, and the revised authenticatable tag is unforgeable.

Theorem 1 (Resistance of data replacing attack and data deleting attack). The improved scheme using revised authenticatable tag can resist data replacing attack and data deleting attack.

Proof. As discussed above, the malicious sever executes its strikes successfully in Tian et al.'s protocol with the help of β_v derived from publicly known authenticatable tag, where

$$\beta_v = (\beta'_v)^{(H_1(id_{q,j}))^{-1}} = u^{H_1(id_{q,j}) \cdot fsk_v \cdot (H_1(id_{q,j}))^{-1}} = u^{fsk_v}.$$

In our improved protocol, the authenticatable tag is generated as $\sigma_{q,j} = (H_2(id_{q,j}) \cdot u^{m_{q,j}})^{fsk_q}$. Considering that the malicious sever wants to forge a valid response, it first attempts to recover β_v from $\sigma_{q,j}$ by computing $\beta_v = u^{fsk_q} = \sigma_{q,j} \cdot H_2(id_{q,j})^{-fsk_q} \cdot u^{-m_{q,j}}$. However, fsk_q is the private key only possessed by each fog node,

which means that it is impossible for the malicious sever to recover β_v from $\sigma_{q,j}$. In other words, the malicious sever cannot launch its strikes anymore in our improved protocol without β_v . $\square$

Theorem 2 (Unforgeability of authenticatable tags). *If the computational Diffie–Hellman (CDH) problem holds in bilinear groups, it is computationally infeasible for any adversary to forge an authenticatable tag.*

Proof. If an unhoneist cloud server can forge a homomorphic verifiable tag with $m'_{i,j}$, i.e. $\sigma'_{i,j} = (H_2(id_{i,j}) \cdot u^{m'_{i,j}})^{ssk_i}$, where $m'_{i,j}$ is a modified data block instead of the raw data block $m_{i,j}$ and $ssk_i = a_i$ is the private key of mobile sink S_i , then there is an adversary who can break a CDH problem instance.

In concrete, to pass the verification phase, we assume that the unhoneist cloud server forges a homomorphic verifiable tag by computing $\sigma'_{i,j} = (H_2(id_{i,j}) \cdot u^{m'_{i,j}})^{ssk_i}$ for a modified data block $m'_{i,j}$, and the original HVT for data block $m_{i,j}$ is $\sigma_{i,j} = (H_2(id_{i,j}) \cdot u^{m_{i,j}})^{ssk_i}$. So we define $\Delta m_{i,j} = m'_{i,j} - m_{i,j}$, and the simulator sets $u = g^{a \cdot h^b}$ for $a, b \in \mathbb{Z}_p^*$. Obviously, we can obtain

$$e(\sigma'_{i,j}/\sigma_{i,j}, g) = e(u^{\Delta m_{i,j}}, g^{a_i}) = e(g^{a \cdot a_i \cdot \Delta m_{i,j}}, g) \cdot e(h^{b \cdot \Delta m_{i,j}}, g^{a_i}) \quad (24)$$

Then, we get

$$\frac{e(\sigma'_{i,j}/\sigma_{i,j}, g)}{e(g^{a \cdot a_i \cdot \Delta m_{i,j}}, g)} = e(h^{b \cdot \Delta m_{i,j}}, g^{a_i}) \quad (25)$$

The above equation can be further written as :

$$\begin{aligned} e(\sigma'_{i,j}/\sigma_{i,j} \cdot g^{-(a_i \cdot a \cdot \Delta m_{i,j})}, g) &= e(h^{b \cdot \Delta m_{i,j}}, g^{a_i}) \\ &= e(h^{a_i}, g^{b \cdot \Delta m_{i,j}}) \end{aligned} \quad (26)$$

Finally, we have $h^{a_i} = (\sigma'_{i,j}/\sigma_{i,j} \cdot g^{-(a_i \cdot a \cdot \Delta m_{i,j})})^{\frac{1}{b \cdot \Delta m_{i,j}}}$.

Due to g is a generator of G and $h \in G$, there exists an integer $z \in \mathbb{Z}_p$ that satisfies $h = g^z$. Therefore, $h^{a_i} = g^{a_i \cdot z}$ is an answer for a CDH problem instance: Given $(g, g^{a_i}, g^z) \in G$, the adversary outputs $g^{a_i \cdot z}$. Note that the answer h^{a_i} will be aborted when $b \cdot \Delta m_{i,j} = 0$ happens. However, b is chosen randomly from $\mathbb{Z}_p^*$ by the simulator, which means the probability of $b \cdot \Delta m_{i,j} = 0$ is only $\frac{1}{p}$. This completes our proof.

$\square$

5. Performance evaluation

In this section, we first give a numerical computational costs analysis of the improved scheme, and then evaluate the performance of the improved scheme by extensive experiments.

5.1. Computational costs

We list all the necessary notations used to describe operations of our improved scheme in Table 1. We analyze the computational cost mainly involved in tag generation phase and proof verification phase. To produce an authenticatable tag for a block, a fog node needs to perform $(2 \cdot E + 1 \cdot M + 1 \cdot H_2)$ operations. It sometimes also has to compute extra signatures that are used by cloud sever to check the integrity of received data blocks, but we don't count this extra computation overhead into the total computational cost, as this step is actually intended for achieving higher security in cloud system. To verify the correctness of the proof, TPA first computes H_v , which brings $(c \cdot E + c \cdot H_2)$ operations. And then TPA conducts $(t \cdot E + t \cdot P + 1 \cdot H_3)$ operations to verify the final equation.

We compare the computational cost in our revised scheme with the basic Tian et al.'s scheme (Tian et al., 2019) and the scheme

Table 1

Notations.

Notation	Operation
M	Multiplication operation on group G
E	Exponentiation operation on group G
P	Pairing operation over group G and G_T
H_1	Hash function that maps a string to an integer in $\mathbb{Z}_p^*$
H_2	Hash function that maps a string to a point on G
H_3	Hash function that maps a point on G_T to a string
c	Number of challenged data blocks
t	Number of fog nodes

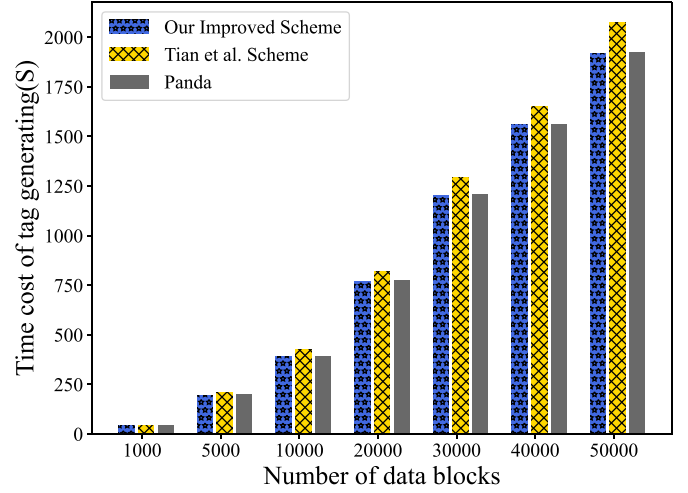


Fig. 2. Time cost of tag generating (Let data block size be 4 KB).

(hereinafter, Panda) in Wang et al. (2015). The protocol named Panda in Wang et al. (2015) considers a data sharing scenario in which all the data blocks are created, modified by multiple users. The nature of the way of auditing data integrity by aggregating challenged blocks belonging to different users in Panda is the same as that in our revised scheme, differing only on the service scenario. Table 2 gives a summary in the comparison of computational costs of abovementioned schemes.

5.2. Experimental results

In this subsection, we evaluate the performance of our improved scheme and give a performance comparison with the other two schemes involved in Table 2. All the experiments are conducted on a personal computer with Intel i7-3770 3.4 GHz processor, 8 GB memory and Ubuntu operating system. In such an ideal environment, we assume that the communication delay between parties does not exist and the error rate of data transmission between parties can be ignored. The Charm-Crypto Library is used in the experiments, and the security parameter is set to be 160 bits. All experiment results are the mean of 15 trials.

We simulate the time cost of tag generation over the three schemes under two different scenarios: with different number of data blocks for various data streams and with different sizes of a data block.

In the first case, by varying the number of data blocks from 1000 to 50,000, where the size of each data block is 4 KB, we simulate the time cost of generating authenticatable tag, and Fig. 2 represents the experimental results. As shown in Fig. 2, the time for processing tag in our improved scheme and Panda is a bit lower than that in basic Tian et al. scheme. For example, for a file with 30,000 data blocks, the time cost in our improved scheme and Panda is about 1203 s and 1207 s, while the fog node in basic Tian

Table 2
Comparison of computational costs.

	Our scheme	Tian et al.'s scheme (Tian et al., 2019)	Panda (Wang et al., 2015)
Tag Generating	$2 \cdot E + 1 \cdot M + 1 \cdot H_2$	$3 \cdot E + 1 \cdot M + 1 \cdot H_1$	$2 \cdot E + 1 \cdot M + 1 \cdot H_2$
Proof Verification	$(t + c) \cdot E + t \cdot P + 1 \cdot H_3 + c \cdot H_2$	$2t \cdot E + t \cdot P + 1 \cdot H_3 + t \cdot H_1$	$4(d + c) \cdot E + (c + 1) \cdot P + c \cdot H_2 + (d + 2c) \cdot M$

Note: d is the number of users who the challenged data blocks belong to. Some integer addition and multiplication operations are omitted, as the running time of those operations is much less than that of other operations.

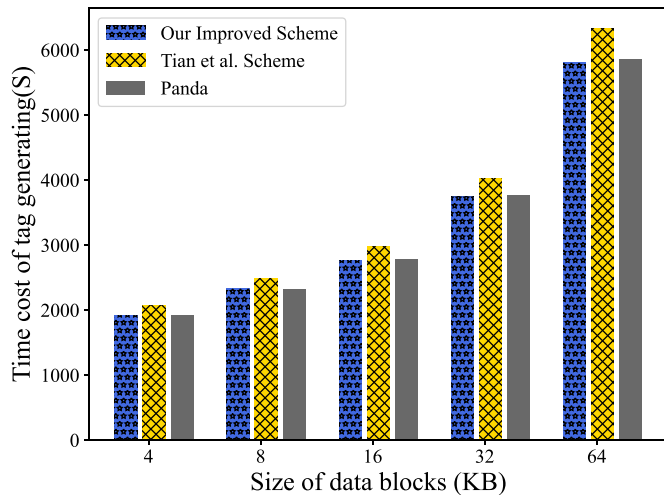


Fig. 3. Time cost of tag generating (Let the number of data blocks be 50,000).

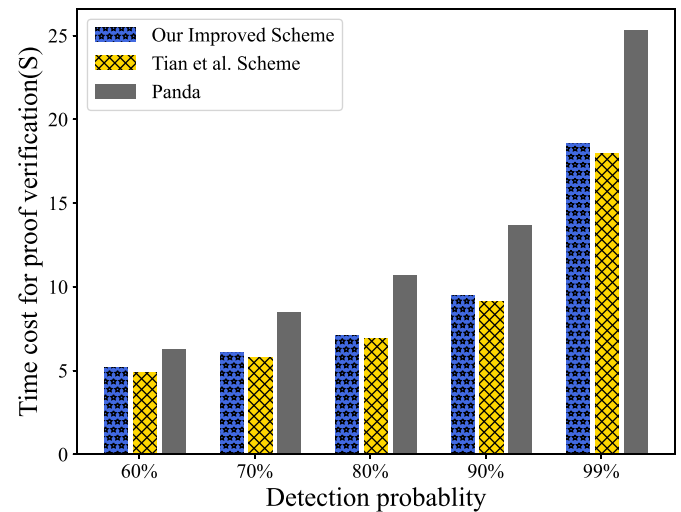


Fig. 4. Time cost for proof verification.

et al. scheme requires 1291 s to conduct such a task. The main reason for this difference is that there is an additional exponentiation operation to be performed in Tian et al. scheme, which is also demonstrated in Table 2. We then change the size of data block to simulate the time cost of tag generating while set the number of data blocks 50,000. Similarly, in Fig. 3, we can see that all the time cost in the three schemes still increases linearly with the size of data block, and the basic Tian et al. scheme takes the most time to generate authenticatable tag. Based on those experimental results, our improved scheme is more practical for time-sensitive IoT scenarios than the basic Tian et al. scheme, such as Augmented Reality, Self-driving, etc.

In our experiments, we select different number of challenged data blocks corresponding to different detection probability to measure the time cost for proof verification. For instance, to achieve 99% detection probability in auditing task with a file that has 1% data error rate, we need to challenge C data blocks each time, in which C satisfies $1 - (1 - 0.01)^C \geq 0.99$. When we set the detection probability 99%, the number of challenged data blocks should be 460. When we set the detection probability 90%, the number of challenged data blocks should be 230 and so forth. The experimental results of time cost at TPA side for detection probability varying from 60% to 99% are shown in Fig. 4. We can learn that for the detection probability 99%, the time cost of TPA in our improved scheme and Tian et al.'s scheme is 18.56 s and 17.94 s, respectively, while the relevant time cost in Panda is 25.3 s. It means that the performance of our improved scheme is improved by 26.6%. The reason is that the size of proof in Panda is far larger than that in other two schemes, which leads to extra exponentiation operations. To put it another way, the communication cost in other two schemes is also significantly reduced compared with Panda.

Remarks. Based on Table 2, Figs. 3 and 4, while toughening the security of Tian et al. (2019), our improved scheme achieves rela-

tively better efficiency. In other words, our new scheme not only repairs the weakness in Tian et al. (2019) and resists the potential attacks, but also keeps the original properties of the basic scheme.

6. Conclusion

In this paper, we perform a cryptanalysis on a tailor-made public auditing scheme in a fog-to-cloud computing environment proposed by Tian et al. and present an improvement on its security. By revealing two potential attacks, we point out the fact that Tian et al.'s scheme fails to achieve soundness as it claimed. Specifically, both data forging attack and data deleting attack launched by malicious cloud sever can convince data owner that all the data is stored correctly and intactly while any data corruption accident cannot be detected by TPA. Therefore, the attacks above make the scheme of Tian et al. (2019) inadequate to meet the security requirements of public auditing schemes. We then designed an applicable authenticatable tag as the countermeasure to those attacks while keeping original properties of the basic scheme. Finally, security analysis and performance evaluation are conducted to show the practicality of our improved scheme.

CRediT author statement

Wenfang Zhang: Conceptualization, Methodology, Writing - original draft, Project administration, Funding acquisition, Resources, Software, Supervision. **Heng Jiao:** Solution design & verification, Data curation, Writing - original draft. **Zhuoqun Yan:** Investigation, Formal analysis, Validation, Visualization. **Xiaomin Wang:** Writing - review & editing, Funding acquisition. **Muhammad Khurram Khan:** Writing - review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

Data will be made available on request.

Acknowledgments

This work was supported in part by the [National Natural Science Foundation of China](#) under Grant [61872302](#); in part by the Key program for Sichuan Science and Technology under Grants 2019YFH0097 and 2019YFG0502.

References

- Ateniese, G., Burns, R., Curtmola, R., Herring, J., Kissner, L., Peterson, Z., Song, D., 2007. Provable data possession at untrusted stores. In: Proceedings of the 14th ACM Conference on Computer and Communications Security, pp. 598–609. doi:[10.1145/1315245.1315318](#).
- Chen, F., Meng, F., Xiang, T., Dai, H., Li, J., Qin, J., 2020. Towards usable cloud storage auditing. *IEEE Trans. Parallel Distrib. Syst.* 2605–2617. doi:[10.1109/TPDS.2020.2998462](#).
- He, K., Chen, J., Yuan, Q., Ji, S., He, D., Du, R., 2021. Dynamic group-oriented provable data possession in the cloud. *IEEE Trans. Dependable Secure Comput.* 1394–1408. doi:[10.1109/TDSC.2019.2925800](#).
- Jiang, T., Chen, X., Ma, J., 2016. Public integrity auditing for shared dynamic cloud data with group user revocation. *IEEE Trans. Comput.* 2363–2373. doi:[10.1109/TC.2015.2389955](#).
- Juels, A., Kaliski, B.S., 2007. PORs: proofs of retrievability for large files. In: Proceedings of the 14th ACM Conference on Computer and Communications Security, pp. 584–597. doi:[10.1145/1315245.1315317](#).
- Li, Y., Yu, Y., Min, G., Susilo, W., Ni, J., Choo, K.-K.R., 2019. Fuzzy identity-based data integrity auditing for reliable cloud storage systems. *IEEE Trans. Dependable Secure Comput.* 72–83. doi:[10.1109/TDSC.2017.2662216](#).
- Shacham, H., Waters, B., 2013. Compact proofs of retrievability. *J. Cryptol.* 442–483. doi:[10.1007/s00145-012-9129-2](#).
- Shen, J., Shen, J., Chen, X., Huang, X., Susilo, W., 2017. An efficient public auditing protocol with novel dynamic structure for cloud data. *IEEE Trans. Inf. Forensics Secur.* 2402–2415. doi:[10.1109/TIFS.2017.2705620](#).
- Shen, W., Qin, J., Yu, J., Hao, R., Hu, J., 2019. Enabling identity-based integrity auditing and data sharing with sensitive information hiding for secure cloud storage. *IEEE Trans. Inf. Forensics Secur.* 331–346. doi:[10.1109/TIFS.2018.2850312](#).
- Sookhak, M., Yu, F.R., Zomaya, A.Y., 2018. Auditing big data storage in cloud computing using divide and conquer tables. *IEEE Trans. Parallel Distrib. Syst.* 999–1012. doi:[10.1109/TPDS.2017.2784423](#).
- Sun, Y., Liu, Q., Chen, X., Du, X., 2020. An adaptive authenticated data structure with privacy-preserving for big data stream in cloud. *IEEE Trans. Inf. Forensics Secur.* 3295–3310. doi:[10.1109/TIFS.2020.2986879](#).
- Tian, H., Nan, F., Chang, C.-C., Huang, Y., Lu, J., Du, Y., 2019. Privacy-preserving public auditing for secure data storage in fog-to-cloud computing. *J. Netw. Comput. Appl.* 127, 59–69. doi:[10.1016/j.jnca.2018.12.004](#).
- Wang, B., Li, B., Li, H., 2014. Oruta: privacy-preserving public auditing for shared data in the cloud. *IEEE Trans. Cloud Comput.* 43–56. doi:[10.1109/TCC.2014.2299807](#).
- Wang, B., Li, B., Li, H., 2015. Panda: public auditing for shared data with efficient user revocation in the cloud. *IEEE Trans. Serv. Comput.* 8 (1), 92–106. doi:[10.1109/TSC.2013.2295611](#).
- Wang, Q., Wang, C., Ren, K., Lou, W., Li, J., 2011. Enabling public auditability and data dynamics for storage security in cloud computing. *IEEE Trans. Parallel Distrib. Syst.* 22 (5), 847–859. doi:[10.1109/TPDS.2010.183](#).
- Zhang, Y., Yu, J., Hao, R., Wang, C., Ren, K., 2020. Enabling efficient user revocation in identity-based cloud storage auditing for shared big data. *IEEE Trans. Dependable Secure Comput.* 608–619. doi:[10.1109/TDSC.2018.2829880](#).
- Zhao, H., Yao, X., Zheng, X., Qiu, T., Ning, H., 2019. User stateless privacy-preserving TPA auditing scheme for cloud storage. *J. Netw. Comput. Appl.* 62–70. doi:[10.1016/j.jnca.2019.01.005](#).

Wenfeng Zhang received the Ph.D. degree in communication and information systems from Southwest Jiaotong University in 2007. She is currently a Full Associate Professor with Southwest Jiaotong University, Chengdu, China. She has authored or co-authored over 70 papers in peer-reviewed journals, proceedings, and book chapters. Her main research interests include the compound signatures, attribute-based cryptography, authentication and key agreement, information security in wireless communications, and cloud computing networks. She is a member of the Chinese Association for Cryptologic Research.

Heng Jiao received the M.S. degree from Southwest Jiaotong University, Chengdu, China, in 2020. His research interests include fog-and-cloud computing and big data security.

Zhuoqun Yan received the B.E. degree from the school of electrical engineering, Zhengzhou university in 2018. Now he is a Ph.D student of the School of Information Science and Technology, Southwest Jiaotong University. His main research interests include digital signatures, security for cloud computing and block-chain.

Xiaomin Wang received the Ph.D. degree in information and communication engineering from Southwest Jiaotong University in 2008. He is currently a Full Professor with Southwest Jiaotong University, Chengdu, China. He has published over 100 research articles in peer reviewed journals and proceedings. He is an inventor of 20 patents. His research interests include information and cyber security, and intelligent maintenance in modern transportation. He is a member of the Chinese Association for Cryptologic Research, the Railway Signal Committee of China Railway Society, and the Sichuan Technical Standard Committee on RailTransit, and a Senior Member of the Automation, Instrument, and Control Society. He received awards for the scientific and technological achievement by the China Railway Society in 2013 and 2019, and the Higher Education Award by the Chinese Association of Automation in 2017.

Muhammad Khurram Khan is currently working as a Professor of Cybersecurity at the Center of Excellence in Information Assurance, King Saud University, Kingdom of Saudi Arabia. He is the Editor-in-Chief of Telecommunication Systems' published by Springer Nature. He is on the editorial board of several journals including, *IEEE Communications Surveys & Tutorials*, *IEEE Communications Magazine*, *IEEE Internet of Things Journal*, *Journal of Network & Computer Applications* (Elsevier), etc. He has published more than 380 papers in the journals and conferences of international reputation. His research areas of interest are Cybersecurity, digital authentication, IoT security, biometrics, multimedia security, cloud computing security, cyber policy, and technological innovation management. He is a fellow of the IET (UK), a fellow of the BCS (UK), and a fellow of the FTRA (Korea). He is the Vice Chair of IEEE Communications Society Saudi Chapter and a distinguished Lecturer of the IEEE. His detailed profile can be visited at <http://www.professorkhurram.com>.